Impactos de desempenho em inteligência artificial e rádio: comparação entre ambientes com e sem Bitdefender

1 Objetivo e limites da comparação

Este relatório compara o desempenho de aplicações de inteligência artificial e operações de arquivos em dois computadores, um declarado sem Bitdefender (baseline) e outro com o produto em execução (Protected). As permissões disponíveis impediram testar o mesmo computador com a proteção ativada e desativada. Por isso, os resultados descrevem a diferença entre os ambientes e não isolam a contribuição exclusiva do Bitdefender. Os processadores, as unidades de armazenamento e os sistemas de arquivos também diferem, conforme a Tabela 1.

Mesmo com essa limitação, observa-se uma tendência de maior duração em várias cargas PyTorch e operações de arquivos, acompanhada de consumo mensurável de CPU pelos componentes do Bitdefender. Também ocorreram melhoras em TensorFlow. A presença dos componentes confirma sua execução, embora a consulta do estado completo da proteção não tenha sido conclusiva.

Tabela 1: Características dos computadores comparados.

	Baseline (sem)	Protected (com)
Processador	Xeon Silver 4316, 20 núcleos / 40 threads	Xeon Gold 5220, 18 núcleos / 36 threads
Frequências	2,30 GHz base / até 3,40 GHz turbo	2,20 GHz base / até 3,90 GHz turbo
Memória	64 GB DDR4, 2 módulos de 32 GB, 2666 MT/s	64 GB DDR4, 4 módulos de 16 GB, 2666 MT/s configurados
Unidade de dados	Samsung 980 PRO, 1 TB	LITEON CA5-8D512, 512 GB
Leitura sequencial	Até 7000 MB/s	Até 3300 MB/s
Gravação sequencial	Até 5000 MB/s	Até 2100 MB/s
Sistema operacional	Linux, Ubuntu	Linux, Ubuntu

As frequências base e turbo são especificações da Intel para o Xeon Silver 4316 e o Xeon Gold 5220, não frequências medidas sob carga. A velocidade nominal dos módulos Protected é 3200 MT/s, com configuração de 2666 MT/s. A Samsung anuncia até 7000 MB/s de leitura e 5000 MB/s de gravação sequencial para o 980 PRO de 1 TB. Taxas do fabricante não representam a duração das operações de arquivos deste ensaio.

2 Bitdefender: recursos ocupados durante o trabalho

A unidade central de processamento (CPU) executa as instruções dos programas. O acelerador gráfico (GPU) pode executar parte dos cálculos dos modelos. Nas medições apresentadas, 100% de CPU corresponde a um processador lógico ocupado. Portanto, 200% significa o equivalente a dois processadores lógicos, não o dobro da capacidade total da máquina. Os 36 processadores lógicos da máquina Protected somam 3600% nessa convenção, sem equivaler necessariamente ao desempenho de 36 núcleos físicos independentes.

Durante os intervalos monitorados da nova campanha, os processos visíveis do Bitdefender consumiram em média 196,30% de CPU, o equivalente a 1,96 processador lógico. O pico de um intervalo foi 617,46%, equivalente a 6,17 processadores lógicos. Isso corresponde a 5,45% da capacidade lógica agregada em média e a 17,15% no intervalo de pico. Esses valores descrevem atividade real do agente durante o ensaio, inclusive preparação e cargas em GPU, e não um recorte exclusivo de treinamento em CPU.

Tabela 2: Componentes na campanha de 5 de novembro. CPU em porcentagem de um processador lógico. Memória residente média em MiB.

Componente	CPU média	CPU pico	Memória
Componente ctc	142,41%	429,97%	1270,13
Processo principal	19,64%	73,98%	365,21
Antimalware	15,65%	246,00%	1255,45
Componente networkmonitor	13,28%	56,45%	125,36
Componente fanotify	5,10%	61,05%	59,61

O componente ctc concentrou 72,55% do tempo de CPU observado do agente. A Tabela 2 mostra que a demanda não se concentra apenas no componente antimalware. A soma da memória residente dos processos teve média de 3,20 GiB e máximo amostrado de 3,23 GiB. Essa soma pode contar páginas compartilhadas mais de uma vez e não mede memória privada exclusiva.

A base histórica do monitor, de 7 e 28 de outubro, cobre 36,82 minutos efetivamente amostrados. Nela, a média ponderada foi 44,29% de um processador lógico e o pico foi 553,41%, equivalente a 5,53 processadores lógicos. Algumas janelas podem refletir limites de recursos aplicados à época. Essa média caracteriza as janelas disponíveis, não um dia típico de uso. Uma medição adicional de 20 segundos, sem limites de CPU aplicados aos serviços, apresentou média de 52,31%.

Nas janelas históricas, a memória contabilizada pelos serviços teve média de 8,79 GiB, enquanto a soma residente dos processos foi de 2,87 GiB. A memória de serviço inclui cache de arquivos e estruturas recuperáveis do kernel. Por isso, interpretar os 8,79 GiB como RAM privada irrecuperável superestimaria essa demanda. Os contadores de entrada e saída por processo estavam indisponíveis na campanha recente. A ausência desses contadores não significa ausência de inspeção de arquivos.

Para uma aplicação que já ocupa quase toda a capacidade de processamento disponível, a demanda do agente reduz a folga para absorver variações. O pico também importa: uma média moderada pode coexistir com intervalos em que vários processadores lógicos atendem ao agente. A consequência depende dos núcleos compartilhados, das filas e dos prazos da aplicação.

3 Resultados de desempenho

Os resultados de treinamento apresentados na Tabela 3 foram obtidos na CPU. Nas cargas PyTorch em CPU apresentadas, Protected levou mais tempo. A rede convolucional com dados gerados passou de 0,9666 s para 1,8065 s, aumento de 86,89%. A taxa correspondente caiu de 6620,85 para 3542,71 amostras/s, redução de 46,49%. Aumento de duração e redução de taxa são formas diferentes de expressar o mesmo trabalho fixo, com denominadores diferentes.

Tabela 3: Durações em segundos. Mediana ± desvio-padrão amostral, três repetições por condição. Variação positiva indica maior duração em Protected.

Tarefa	Baseline (sem)	Protected (com)	Variação
CPU e operações de arquivos			
Inicialização do PyTorch	0,7451 ± 0,0182	1,8406 ± 0,0526	+147,04%
Treino de rede convolucional / PyTorch	0,9666 ± 0,0773	1,8065 ± 0,0580	+86,89%
Treino de linguagem / PyTorch	13,8190 ± 0,4633	19,8194 ± 0,5534	+43,42%
Treino de rede convolucional / TensorFlow	1,5161 ± 0,3640	0,7959 ± 0,0120	-47,51%
Criação e leitura de 1024 arquivos	0,0407 ± 0,0071	0,4816 ± 0,0192	+1084,35%
10000 leituras aleatórias	0,0191 ± 0,0001	0,3091 ± 0,0716	+1518,53%
Gravação e leitura de controle, 64 MiB	0,0536 ± 0,0054	0,3279 ± 0,0753	+511,36%
Gravação persistente de modelo / PyTorch	0,0224 ± 0,0001	0,0307 ± 0,0049	+37,25%
Avaliação de 2000 exemplos / regressão logística	0,0015 ± 0,0002	0,0054 ± 0,0024	+260,25%

O caso TensorFlow em CPU com dados gerados apresentou duração 47,51% menor em Protected. Com CIFAR-10, porém, a rede convolucional TensorFlow levou 28,37% mais tempo. A direção das diferenças depende da carga e do ambiente. Uma penalidade única para todo treinamento não representa esses resultados.

Nas operações de pequenos arquivos, o aumento foi de 40,67 ms para 481,62 ms por conjunto de 1024 gravações e 1024 leituras. São aproximadamente 441 ms adicionais por conjunto, não por arquivo. As 10000 leituras aleatórias passaram de 19,10 ms para 309,14 ms. Os controles usam arquivos recém-criados e podem ser atendidos parcialmente pelo cache, portanto não medem a velocidade física máxima das unidades.

Checkpoint de modelo é um arquivo que preserva o estado do modelo e do otimizador para restaurar o treinamento. Sua gravação persistente em PyTorch aumentou 37,25%. Já a variante sem sincronização persistente ficou 25,81% mais rápida em Protected. O controle de 64 MiB da tabela grava, sincroniza e lê um arquivo comum. Ele não serializa um modelo e não deve ser usado como estimativa universal do custo de checkpoints.

Cada valor reúne três repetições concluídas da mesma sessão. O desvio-padrão descreve a dispersão dessas repetições e não é um intervalo de confiança. As cargas de treinamento selecionadas usam 200 passos medidos e cinco de aquecimento. Foram comparadas tarefas com os mesmos parâmetros, conteúdo dos conjuntos de dados e versões das bibliotecas. A comparação é descritiva entre máquinas.

4 O que essas ordens de grandeza significam em IA

4.1 Treinamento: um pequeno custo pode se repetir milhões de vezes

Treinar um modelo significa ajustar seus parâmetros usando sucessivos lotes de exemplos. Uma época corresponde a percorrer o conjunto de treinamento uma vez. Portanto, uma diferença pequena por lote ou por leitura pode se repetir muitas vezes. Milissegundos e meses se relacionam pela quantidade de repetições e pela parcela do tempo realmente afetada.

Considere, como exemplo, um treinamento com 14 milhões de amostras, organizadas em lotes de 32 exemplos. Cada época exigiria 437500 lotes. Um atraso adicional de 1 ms por lote somaria 437,5 s, ou 7,29 minutos por época. Em 100 épocas, seriam 12,15 horas adicionais. Esse cenário hipotético mostra como um custo pequeno por operação pode se acumular ao longo do treinamento.

Para tornar um treinamento de um mês concreto, considere uma duração original de 30 dias. Se uma etapa que ocupa todo esse tempo passasse a custar 86,89% mais, como no caso PyTorch em CPU da tabela, a projeção seria 56,07 dias. Se apenas 20% do tempo original pertencesse à etapa afetada, a projeção cairia para 35,21 dias. Essas projeções ilustram o efeito acumulado de uma etapa mais lenta. A duração real depende da parcela do treinamento afetada, e os percentuais observados entre os computadores não isolam o efeito do Bitdefender.

$$T_{\text{projetado}} = T_{\text{original}}(1 + f\,d), \tag{1}$$

em que f é a fração do tempo original afetada e d é o aumento relativo dessa etapa, em forma decimal. O exemplo usa $d = 0{,}8689$. Sobreposição entre processamento e leitura, cache e execução em GPU podem mudar a fração afetada e exigem medição própria.

4.2 Inferência: medir o modelo também exige medir o ambiente

Inferência é usar um modelo já treinado para produzir uma resposta. Neste ensaio, a regressão logística apresentou 1,49 ms em baseline e 5,37 ms em Protected para avaliar um conjunto de 2000 exemplos. A rotina inclui a predição e o cálculo da acurácia. Esses tempos não são a latência de uma única inferência. A dispersão de 2,39 ms em Protected também mostra por que poucas medidas isoladas podem representar mal uma comparação.

Para um artigo, uma tarefa científica idêntica pode parecer mais lenta quando a medição inclui disputa com serviços de fundo. A consequência é uma comparação menos reprodutível ou uma conclusão sobre eficiência que o experimento não sustenta. Deve-se registrar hardware, carga concorrente e política de proteção, repetir as medidas e comparar modelos sob condições equivalentes. A diferença de ambiente pode ser confundida com uma diferença entre modelos. As medidas apresentadas referem-se à avaliação de um conjunto de exemplos, e uma análise de latência por resposta exige um ensaio específico.

5 Rádio: capacidade média e cumprimento de prazos

Em rádio definido por software, parte do processamento do sinal é executada pelo computador. As amostras chegam continuamente e precisam ser tratadas em tempo hábil. Aumentar a taxa de amostras ou a complexidade do processamento aproxima a aplicação do limite de CPU. Nesse regime, o trabalho concorrente pode reduzir a taxa sustentável ou provocar acúmulo de dados e perdas, dependendo da margem dos buffers.

Em um rádio operando a 8 milhões de amostras por segundo, pequenas pausas já correspondem a grandes quantidades de dados. Como exemplo de ordem de grandeza, 1 ms nessa taxa corresponde a 8000 amostras recebidas e 500 ms correspondem a 4 milhões de amostras. Se uma pausa impedisse toda a cadeia de processar dados durante esse intervalo, seria necessário armazenar esse volume e depois recuperar o atraso. Nesse exemplo, os números traduzem a duração de uma pausa na quantidade de amostras que precisaria aguardar processamento.

Considere ainda um estágio hipotético que recebe um bloco novo a cada 1 ms e leva 0,95 ms para processá-lo. Sua folga é de apenas 0,05 ms. Se o custo por bloco aumentar 10%, o processamento passa a 1,045 ms, superior ao intervalo de chegada. A fila cresce enquanto essa condição persistir. Assim, uma diferença de apenas 0,095 ms por bloco pode comprometer a operação contínua mesmo que pareça desprezível em uma tarefa isolada.

5.1 Por que isso pode aparecer como internet mais lenta

Se a capacidade de processamento limita o enlace, reduzir o trabalho disponível por segundo pode exigir uma taxa de dados menor para manter o fluxo estável. Em um modelo ilustrativo de capacidade perfeitamente divisível, retirar 5,45% da capacidade de uma aplicação já saturada transformaria um limite de 100 Mbit/s em cerca de 94,55 Mbit/s. O percentual vem da ocupação média agregada do agente nesta campanha, mas a conversão em taxa de dados pressupõe escala linear e disputa pelos mesmos recursos. Ela não é uma medição de internet ou de rádio.

O pico observado, equivalente a 6,17 processadores lógicos, evidencia a importância da margem para variações. Sua duração e o escalonamento determinam se há aumento de latência, absorção pelos buffers ou perda de dados. Não é correto converter esse pico diretamente em uma queda sustentada de 17,15% na taxa de dados. A validação necessária é executar o rádio na taxa pretendida e registrar perdas, filas, latência e utilização por núcleo junto com o agente.

6 Conclusão

A comparação mantém uma tendência desfavorável em várias tarefas PyTorch e de arquivos, com exceções favoráveis em TensorFlow. O dado mais direto sobre o Bitdefender é sua demanda concorrente: média de 1,96 processador lógico e pico amostrado de 6,17 durante a campanha. Para IA, o efeito prático depende de quantas vezes a etapa se repete. Para rádio, depende de a execução terminar antes da chegada do próximo bloco. As diferenças entre os PCs impedem separar uma penalidade causal exclusiva da proteção.

Os resultados comparam medições de 3 de novembro de 2026 no ambiente sem Bitdefender e de 5 de novembro no ambiente com Bitdefender. Foram consideradas somente execuções concluídas sem avisos ou indícios registrados de interferência.